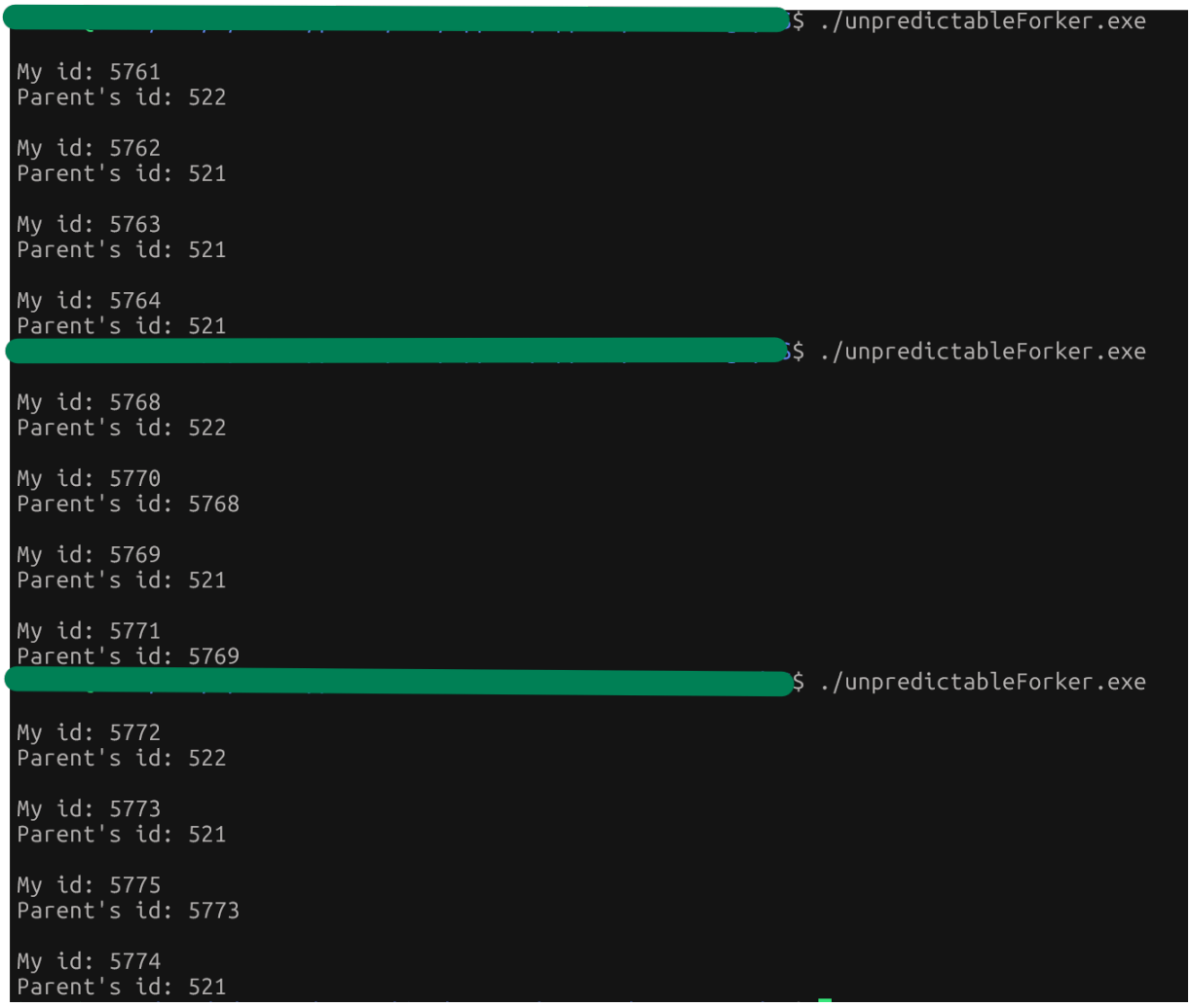
Individual Section

Code Deliverable: unpredictableForker.cpp

```
// i/o
#include <stdio.h>
// pid
#include <wait.h>
// uint types
#include <stdint.h>

int main(int argc, char* argv[])
{
	fork();
	fork();
	fprintf(stdout, "\nMy id: %llu\nParent's id: %llu\n", (uint64_t)getpid(), (uint64_t)getppid());
	return 0;
}
```

Executable Output



PID Table

Run #	Output Lines	Order	Notes
1	My id: 5761 Parent's id: 522 My id: 5762 Parent's id: 521 My id: 5763 Parent's id: 521 My id: 5764 Parent's id: 521	Parent Child 1 Child 2 Child 3	The processess processed in the order of their creation. However, the children of 5761 are not childed to 5761 at output. 5761 must have terminated before its children and grandchildren's printf() call went through. This means the children got orphaned before getting their new parent, 521.
2	My id: 5768 Parent's id: 522 My id: 5770 Parent's id: 5768 My id: 5769 Parent's id: 521 My id: 5771 Parent's id: 5769	Parent Child 2 Child 1 Child 3	The processess did NOT output in order of creation. Child 2 printed before child 1. Child 2, 5770, prints before the original parent process, 5768, terminates. Child 2, 5770, was the second child created by the parent. There is no other way for it to be parented to 5768. Other children are orphaned before they print. When Child 1, process 5769, prints, it is childed to process 521. When Child 3, process 5771, prints, it is childed to Child 1, process 5769. Child 1 forked to create Child 3.
3	My id: 5772 Parent's id: 522 My id: 5773 Parent's id: 521 My id: 5775 Parent's id: 5773 My id: 5774 Parent's id: 521	Parent Child 1 Child 3 Child 2	The processes did NOT output in order of creation. Parent and Child 1 printed in order. Child 3 printed before Child 2. Every child was orphaned before it printed to console. Child 3 was created from Child 1. That's the only way it could be childed to Child 1. Therefore, Child 2 was created by the original parent. Child 1 and 2 were childed to process 521 after the original parent's termination. I'm wondering if process 521 functions as my wsl's init (or systemd) process? Init (or systemd) usually has a pid of 1. I am not sure.

Group Section

Each member shares their **3-run table** with the group. Group answers 5 questions below and creates **a process tree diagram**:

1. Did all members see **the same number of output lines**?
    1. No, some group members had 4 output lines while, at least one, had 8 output lines. This is because some output the pid and ppid on the same line while others output on different lines.
2. Did orders differ between group members?
    1. Yes. The order which processes are processed are non-deterministic. The different programs on our different machines processed in different orders.
3. What do they notice about **parent and child PID relationships**?
    1. Child PIDs are chosen by incrementing from the last chosen PID from their parent/sibling. The parent's PID will always be the lowest number.
    2. You can deduce which process created what based on PID states at output.
4. Are there any **unexpected results**?
    1. Sometimes, the child is parented to a different process than the parent that spawned them. This suggests the parent process terminated before the child displayed their output.
    2. I wasn't expecting the parent to consistently execute first.
5. Process Tree Diagram
    1. 521
        1. 5762
        2. 5763
        3. 5764
        4. 5769
            1. 5771
        5. 5773
            1. 5775
        6. 5774
    2. 522

1. 5761
2. 5768
    1. 5770
3. 5772

Member Name	Peter Doria	Joshua Archer	Allen Le	Yadid Alailla
Role	Member	Member	Member	Member
Tasks Completed	1-3	1-3	1-3	1-3
Challenges Faced	Learning to deduce child creation order and relationships from output. It was fun!	Figuring out differences in behavior between nested and non-nested fork being required.		
Achievements	Provided C language resources to the group (i.e. Beej's Guide to C, fork() linux manual reference).	Had a fun discussion regarding child and parent processess.		
Contribution & Performance Rates	5	5	5	5